PantryPilot

Final Project Presentation

Smart meal and grocery planning with executable, safety-aware quality controls

System

Meal plans, substitutions, pantry use, and grocery lists

Quality focus

Risk, CM, traceability, and system-level testing

Prepared by

Zixuan Liang
CISC 594

Assignment coverage map

1 — Project introduction and system features

2 — Role of each group member

3 — Risk identification and tracking

4 — Configuration management approach

5 — System testing approach and outcome

Central message

PantryPilot combines useful automation with disciplined SQA: risks are identified, controls are versioned and tested, and the same evidence is visible in a live web demo.

Project introduction: why PantryPilot exists

Planning friction

Users often decide meals, check ingredients, and build grocery lists in separate tools.

Constraint complexity

Allergies, disliked ingredients, budgets, nutrition goals, and preferences can conflict.

Food waste

Pantry items expire while shopping decisions ignore what is already available at home.

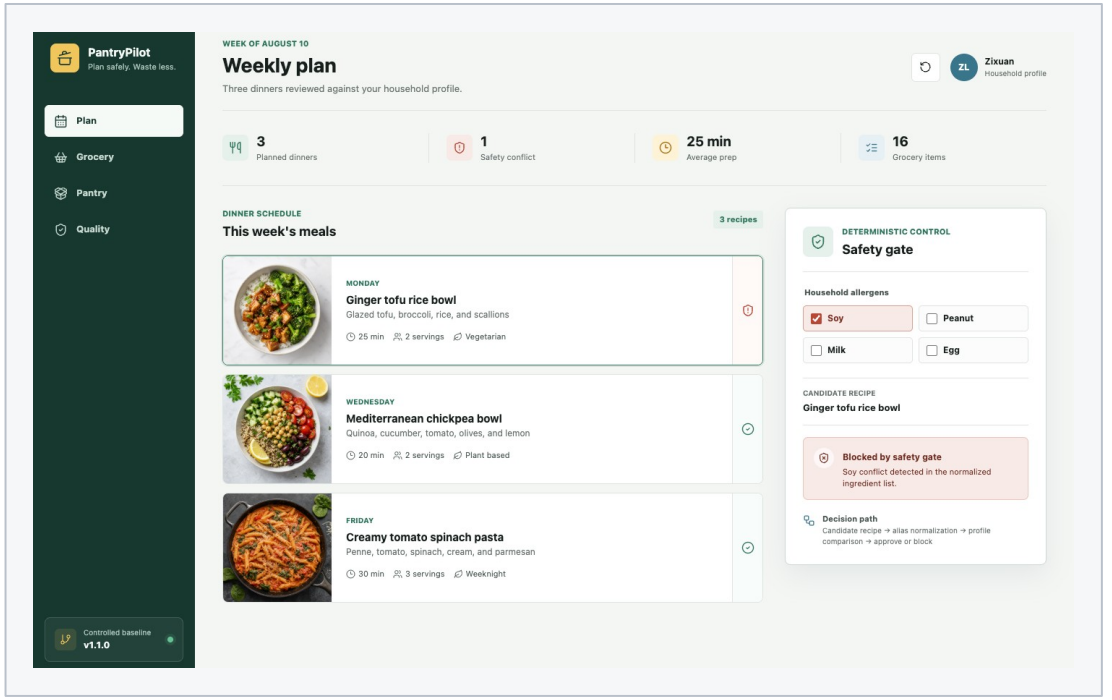
PantryPilot answer

A smart meal and grocery planning system that maintains profiles, recipes, pantry inventory, weekly meal plans, AI substitutions, and consolidated grocery lists.

System features and live demonstration

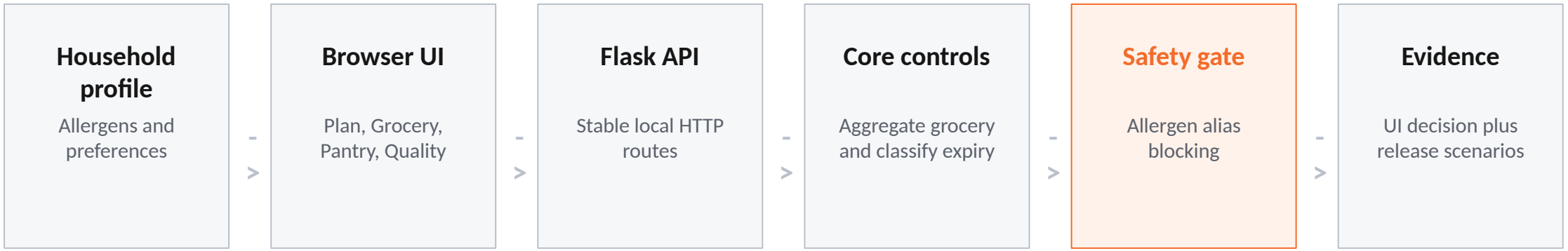
Controlled v1.1.0 baseline

- Deterministic allergen conflict detection
- Grocery aggregation by canonical name and unit
- Inclusive pantry-expiry boundary logic
- Release identity and required quality gates
- Six repeatable release-level scenarios



Live demo path: Plan safety gate -> Grocery aggregation -> Pantry expiry -> Quality evidence.

Executable design: browser to deterministic controls



Design decision

The browser never decides safety; Flask delegates to deterministic, test-covered functions.

Role of each group member

This was completed as an individual project, so Zixuan Liang owned each presentation section and project artifact.

Project and requirements

Defined the PantryPilot concept, V1/V1.1 scope, requirements, adverse conditions, and quality goals.

Risk management

Built and updated the risk table, likelihood-impact scores, mitigation strategy, and weekly tracking history.

Configuration management

Consolidated the project into GitHub, preserved tags/history, controlled the Flask UI files, and updated the CI and CM evidence.

System testing

Implemented the web demo, six release scenarios, five Flask endpoint tests, responsive browser QA, and the final test report.

Risk identification and tracking method

Identification

Risks were identified from safety-sensitive outputs, external dependencies, optimization complexity, data handling, and release scope.

Scoring

Each risk received a probability score and an impact score from 1 to 5; total priority equals probability times impact.

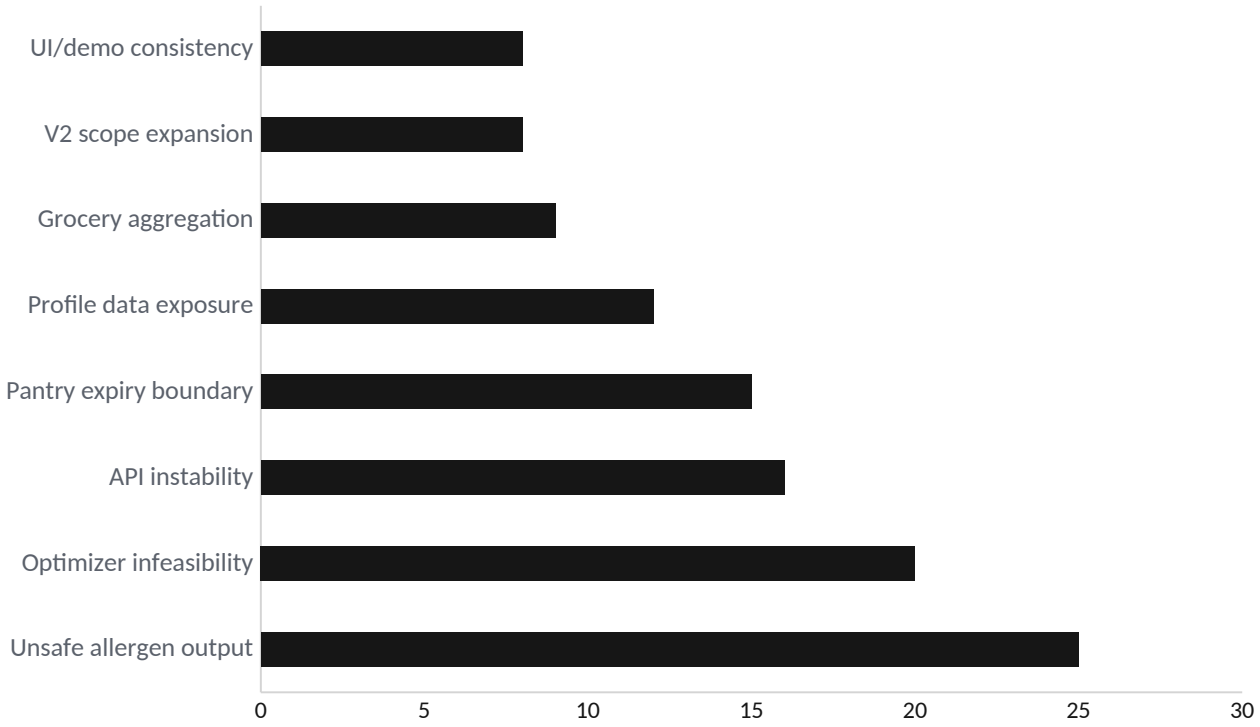
Tracking

Risk status was reviewed weekly and linked to controls, tests, CI checks, release notes, or retirement rationale.

Current risk posture

R1-R5 remain active; R6 and new R8 UI/demo consistency are monitored; R7 scope expansion remains retired/controlled.

Top project risks by score



Priority conclusion

- Highest: unsafe LLM output to an allergic user
- Critical: optimizer infeasibility or runtime
- R8 UI/demo consistency: 8 (medium)
- Five web tests keep the backend as source of truth

How the main risks were reduced

Risk	Reduction control	Verification evidence
Unsafe allergen output	Deterministic allergen filter blocks conflict before display	System test V1.1-ST-01 returned safe=false for soy conflict
Optimizer failure	Bounded solver concept and explicit future relaxation order	Kept out of V1.1 release baseline
Recipe API instability	Offline fixtures/cache strategy and normalized ingredient model	System tests avoid external network dependency
Pantry expiry boundary	Inclusive 3-day rule; expired items excluded	V1.1-ST-03 passed today/+3/+4/yesterday boundary checks
UI/demo consistency	Flask API is source of truth; assets are local; demo state is resettable	WEB-01 through WEB-05 pass; desktop/mobile browser review completed

Control rule: each major risk requires a design control and repeatable evidence.

Configuration management approach

1

Main baseline
Stable release branch

->

2

Feature branch
feature/safety-grocery-controls

->

3

Pull request

Scope, tests, and review evidence

->

4

CI gate

11 pytest tests plus 6 system scenarios

->

5

Release tag

v1.1.0 tagged; web demo [Unreleased]

Controlled items

Backend and Flask source, template, JavaScript, CSS, local assets, tests, requirements, CI workflow, changelog, reports, and release records.

Configuration management evidence

Evidence	Project record
Repository package	github.com/liangzixuan/cisc-594 / PantryPilot_CM_Generic_Files
Branches	main; feature/safety-grocery-controls; full history preserved in root repo
Release tags	v1.0.0 and v1.1.0 tagged; Flask demo recorded under [Unreleased]
Key commits	842ae5e baseline; 72a997e feature; 834e64e release; 51e7a65 history merge
CI workflow	.github/workflows/pantrypilot-ci.yml runs pytest and system_test_runner.py
Traceability	CHANGELOG, release notes, tests, updated reports, and Git history preserve traceability

System testing approach

Methodology

Six release scenarios cover core behavior; five Flask test-client cases verify routes, requests, decisions, and evidence reporting.

Environment

macOS 26.5.1, Python 3.11.15, CI Python 3.12, Flask 3.1.1, pytest 8.2.2, Chrome 150.0.7871.129.

Setup

Install requirements; run pytest and system_test_runner.py; start the demo with python -m src.web_app.

Test class	Coverage target
Release readiness	Version identity and required quality gates
Core + web/API workflow	Grocery aggregation plus Flask route, request, response, and quality-summary checks
Safety behavior	Allergen conflict blocked; safe recipe allowed
Boundary conditions	Pantry item expires today, +3 days, +4 days, and yesterday

System testing outcome

11/11

pytest tests passed

6 / 6 formal release scenarios also passed; no defects were observed.

Scenario	Actual result
V1-ST-01 release identity	PantryPilot 1.1.0: Safety & Grocery Controls
V1-ST-02 quality gates	ci_required, pull_request_required, review_required, unit_tests_required
V1-ST-03 grocery aggregation	rice quantity 3.0 cup
V1.1-ST-01 allergen conflict	soy conflict detected; safe=false
V1.1-ST-02 safe recipe	safe=true
V1.1-ST-03 expiry window	today=true, +3=true, +4=false, yesterday=false

Final takeaway and references

PantryPilot's quality story is visible in working software, not just documents.

The web demo makes the same deterministic safety decisions, versioned controls, risk mitigations, and test evidence inspectable in one flow.

Project sources	- Updated risk, CM, and system testing reports - CISC-594 GitHub repository and CI workflow - PantryPilot Flask demo and local visual assets - 11 pytest tests plus 6 release scenarios

Questions